



---

Instructor: **Dr. Ali Imran Sandhu**  
Undergraduate Course: **Artificial Intelligence**  
**Due Date:** 12PM, Oct 9, 2026

Assignment Number: **1**  
Date: October 3, 2026  
Mapping: **CLO 1 - GA 2 - C2**

---

## Why this assignment

By the end, you will be able to explain *why*:

- (a) MSE is the natural loss for regression.
- (b) Cross-entropy is the natural loss for classification.
- (c) Cross-entropy, KL divergence, and negative log-likelihood are the same idea seen from different angles.

Each part builds on the previous one, so please do not skip ahead. Hints are marked *Hint*. Try each problem for at least 10 minutes before reading a hint.

## The Big Picture

Every machine learning model must be trained by minimizing a *loss function*, a number that measures how badly the model's predictions disagree with the data. Two questions look simple but are rarely explained: (i) why do we use the squared error for *regression* (predicting a number, such as house price), and (ii) why do we use *cross-entropy* for *classification* (predicting a category, such as cat, dog, or bird)? Why not use the squared error  $(y - \hat{y})^2$  everywhere?

The answer comes from probability theory. Real data is noisy and uncertain, so a model should not just output an answer, it should describe a *probability distribution* over possible answers. Once we do this, the loss is no longer a guess; it follows from one principle, **maximum likelihood**: choose the parameters that make the observed data most probable. This assignment builds that story step by step, from basic probability to entropy, cross-entropy, KL divergence, and finally to the two losses.

## Submission instructions (read carefully)

1. **Handwritten only.** Typed, printed or tablet-generated work is not accepted.
  2. **Neat, clean and crystal clear.** Use a dark pen on white paper and label every part (e.g. 3(c)).
  3. **Show every step.** Answers without working receive no credit.
  4. **Scanning.** Scan each page with a scanning app such as CamScanner. Only sharp, well-lit, precisely focused, flat and upright scans are accepted; blurred, dark, angled or cropped pages are rejected.
  5. **One PDF,** with pages in order and your name and ID on the first page.
  6. **Automated initial check.** All submissions are first checked by language models. If your work cannot be read, it cannot be checked.
  7. **Messy, illegible or poorly scanned work will be penalised and may not be graded at all.**
-

## Part 1: Probability Basics

**Goal:** Get comfortable with probabilities of events and independence.

**Why this matters in machine learning.** A trained classifier does not simply say “this is a cat”; its final layer outputs probabilities such as  $(0.85, 0.10, 0.05)$  for (cat, dog, bird). These numbers must be non-negative and sum to 1, exactly like a probability distribution. Also, when a dataset contains many training examples, we almost always assume they were collected *independently* of each other (the “i.i.d.” assumption). Independence is what lets us compute the probability of the whole dataset by *multiplying* the probabilities of individual examples, which is the starting point for the likelihood in Part 4.

**1.1** A fair six-sided die is rolled once. Write the full probability distribution (one probability per outcome). What must the probabilities add up to?

**1.2** A biased coin has  $P(\text{heads}) = 0.7$ . You flip it twice. Compute:

- (a)  $P(H, H)$
- (b)  $P(H, T)$
- (c)  $P(\text{exactly one head in two flips})$

*Hint:* Independent events multiply. For (c), list all the ways to get exactly one head, then add.

**1.3** In your own words, what does it mean for two events to be independent? Give one example of events that are independent and one example of events that are not.

## Part 2: Random Variables, Expectation, Variance

**Goal:** Learn to summarize a distribution with numbers.

**Why this matters in machine learning.** A random variable is a quantity whose value is uncertain, such as the price of a house given its size, or the class of an image. The *expectation*  $E[X]$  is the long-run average value, and the *variance* measures how spread out the values are around that average.

Both ideas appear directly in training. The loss over a dataset is an average, which is an expectation of the per-example loss. In regression, a model with prediction  $\hat{y}$  is really estimating the *expected* value of  $y$  given the input  $x$ , and the variance describes how noisy the data is around that prediction. Notice that variance is built from a *squared* difference; this is not a coincidence, and you will see the same squared difference reappear as the MSE loss in Part 5.

**2.1** A random variable  $X$  takes values 1, 2, 3 with probabilities 0.2, 0.5, 0.3. Compute the expectation  $E[X] = \sum_x x P(x)$ .

**2.2** Compute the variance  $\text{Var}(X) = E[(X - E[X])^2]$  for the same  $X$ .

*Hint:* Compute  $E[X]$ , subtract it from each value, square, then take the weighted average using the same probabilities.

**2.3** Why do we use the *squared* difference in the variance and not just  $(X - E[X])$ ? What would  $E[X - E[X]]$  equal? (You will meet squared differences again in Part 5. Keep this thought.)

---

## Part 3: Three Key Distributions

**Goal:** Know the three distributions that appear in ML losses.

**Why this matters in machine learning.** Different prediction problems produce different kinds of outputs, and each kind has its own natural probability distribution:

| Type of problem                        | What is predicted        | Natural distribution            |
|----------------------------------------|--------------------------|---------------------------------|
| Binary classification (spam or not)    | one of 2 outcomes        | Bernoulli                       |
| Multi-class classification (digit 0–9) | one of $K$ outcomes      | Categorical (output of softmax) |
| Regression (temperature, price)        | a real number with noise | Gaussian                        |

Choosing the loss function for a problem is therefore the same as choosing which distribution describes the target  $y$ . Once you write down the formula of the distribution, the loss follows automatically (Parts 4, 5, and 7). This part is about becoming fluent with those three formulas.

**3.1 Bernoulli.** A single coin flip with  $P(1) = p$  and  $P(0) = 1 - p$ . Show that this can be written in one formula:

$$P(y) = p^y(1 - p)^{1-y}.$$

Check it by plugging in  $y = 1$  and  $y = 0$ .

**3.2 Categorical.** A  $K$ -sided die has probabilities  $p_1, \dots, p_K$ . A label is written as a *one-hot vector*  $\mathbf{y}$ ; for example  $(0, 1, 0)$  means “class 2”. Show that

$$P(\mathbf{y}) = \prod_{k=1}^K p_k^{y_k}$$

gives the correct probability for the one-hot vector  $(0, 1, 0)$  with 3 classes.

*Hint:* Anything raised to the power 0 is 1. Only one term in the product survives.

**3.3 Gaussian (normal).** The density of a Gaussian with mean  $\mu$  and variance  $\sigma^2$  is

$$p(y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right).$$

(a) Sketch this curve for  $\mu = 0$ . Where is it highest?

(b) Without calculating, which has a higher density:  $y = \mu$  or  $y = \mu + 3\sigma$ ? Explain what this says about “noise”.

## Part 4: Likelihood and Maximum Likelihood Estimation (MLE)

**Goal:** Learn to choose model parameters that make the observed data most probable.

### Problem statement.

Suppose a model has parameters  $\theta$  (for example, the weights of a neural network, or the probability  $p$  of a coin). Given the parameters, the model assigns a probability to every possible dataset. We have observed one particular dataset, so we ask: *which parameters make this observed dataset most probable?* The function that answers this, viewed as a function of  $\theta$  with the data fixed, is called the **likelihood**, and choosing  $\theta$  to maximize it is called **maximum likelihood estimation (MLE)**.

### Connection to machine learning.

*Training a model is maximum likelihood estimation.* For a dataset of  $n$  independent examples, the likelihood is a product of  $n$  probabilities, which becomes extremely small and numerically unstable.

So we take the logarithm, which turns the product into a sum, and we flip the sign so that maximizing becomes minimizing. The result is the **negative log-likelihood (NLL)**, and this is the loss function that we hand to gradient descent. In Parts 5 and 7 you will see that MSE and cross-entropy are both simply the NLL under different assumed distributions. The coin-flip problem below is the smallest possible example of the whole procedure.

**Key idea:** *Probability* asks “given the parameters, how likely is this data?” *Likelihood* uses the same formula, but we view it as a function of the parameters while the data stays fixed.

**4.1** You flip a coin 8 times and see 3 heads and 5 tails. Let  $p = P(\text{heads})$ .

- (a) Write the likelihood  $L(p)$ .
- (b) Fill in this table:

|        |     |     |       |     |     |
|--------|-----|-----|-------|-----|-----|
| $p$    | 0.1 | 0.3 | 0.375 | 0.5 | 0.8 |
| $L(p)$ |     |     |       |     |     |

- (c) Which  $p$  gives the largest  $L$ ? Does it match your intuition?

*Hint for (a):* Each head contributes a factor  $p$ , each tail a factor  $(1 - p)$ . Flips are independent.

**4.2** Why do we usually work with the **log** of the likelihood? Give two reasons.

*Hint:* Think about (i) multiplying 1000 numbers each smaller than 0.1, and (ii) what the log does to a product.

**4.3 Derive the MLE.** For  $k$  heads in  $n$  flips:

- (a) Write the log-likelihood  $\ell(p) = k \log p + (n - k) \log(1 - p)$ .
- (b) Differentiate with respect to  $p$  and set the result to zero.
- (c) Solve for  $p$ . You should obtain  $\hat{p} = k/n$ .
- (d) Substitute  $k = 3$ ,  $n = 8$  and compare with your table in 4.1.

*Hint:*  $\frac{d}{dp} \log p = \frac{1}{p}$  and  $\frac{d}{dp} \log(1 - p) = -\frac{1}{1 - p}$ .

**4.4 Negative log-likelihood.** Optimizers minimize a loss. Explain why “maximize  $\ell$ ” is the same as “minimize  $-\ell$ ”.

## Part 5: Gaussian Noise Leads to MSE

**Goal:** Discover where MSE comes from.

### Problem statement.

In regression we have inputs  $x_i$  and real-valued targets  $y_i$ , and we fit a model such as the line  $\hat{y} = wx + b$ . Real data never lies exactly on the line, so we must say something about the leftover scatter. The standard modelling assumption is

$$y_i = wx_i + b + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

that is, the target equals the model’s prediction plus random Gaussian noise. Equivalently, the model predicts the *center* of a bell curve, and the actual  $y_i$  is drawn from that bell curve.

### Connection to machine learning.

Under this assumption, the negative log-likelihood of the data turns out to be the **sum of squared errors**. In other words, mean squared error is *not* an arbitrary choice; it is what maximum likelihood produces when the noise is Gaussian with constant spread. This also tells you when MSE is a poor

choice: if the noise has heavy tails or outliers, the Gaussian assumption is wrong, and other losses (for example, absolute error, which corresponds to Laplace noise) may be more appropriate. First you will do the simplest case, estimating one mean from three measurements, and then extend to a regression line.

**5.1 Estimating a single mean.** You measure a quantity three times and get 2, 4, 9. Assume each measurement equals  $\mu$  plus Gaussian noise with known  $\sigma$ .

- Write the likelihood  $L(\mu)$  as a product of three Gaussian densities.
- Take  $-\log L$ . Show that it equals (constants)  $+\frac{1}{2\sigma^2} \sum_i (y_i - \mu)^2$ .
- Which terms do not depend on  $\mu$ ? Why can we ignore them when finding the best  $\mu$ ?
- Minimize  $\sum_i (y_i - \mu)^2$  with respect to  $\mu$ . Confirm that  $\hat{\mu} = 5$ .
- Compute the sum of squares at  $\mu = 5$  and at  $\mu = 4$ . Which is smaller?

*Hint for (b):* The log of a product is a sum of logs, and  $\log(e^a) = a$ .

*Hint for (d):* Differentiate and set to zero. You will get  $-2 \sum_i (y_i - \mu) = 0$ .

**5.2 From a mean to a regression line.** In linear regression, the “true value” changes with  $x$ :

$$y_i = wx_i + b + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

- What is the distribution of  $y_i$  given  $x_i$ ? State its mean and variance.
- Write the negative log-likelihood of  $n$  data points.
- Show that minimizing it is the same as minimizing

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - (wx_i + b))^2.$$

**5.3 Reflection (3 to 4 sentences).** Explain to a friend: “MSE is not arbitrary; it comes from an assumption.” What is that assumption? When might it be wrong?

## Part 6: Information, Entropy, Cross-Entropy, KL Divergence

**Goal:** Measure “surprise” and “mismatch between distributions”. Use  $\log_2$  (answers in *bits*) unless told otherwise.

**Problem statement.** In classification, both the truth and the model’s prediction are *probability distributions* over the classes. The true label of an image is a distribution that puts probability 1 on the correct class (a one-hot vector, e.g.  $(0, 1, 0)$ ), and the network outputs a predicted distribution (e.g.  $(0.2, 0.7, 0.1)$ ). We need a single number that measures how far apart these two distributions are. Simply computing  $y - \hat{y}$  is not satisfactory, because it ignores the fact that we are comparing distributions. Information theory provides the right tools, built in four steps below.

**(a) Surprise (information).** The *surprise* of an event with probability  $p$  is  $-\log p$ . Rare events are very surprising, and certain events carry no surprise. In machine learning, the loss for a single training example is exactly the surprise the model felt when it saw the true label: if the model gave the correct class probability 0.001, it was extremely surprised and receives a large loss.

**(b) Entropy  $H(p)$ .** *In general:* entropy is the average surprise of a distribution  $p$ ,  $H(p) = -\sum_k p_k \log p_k$ . It measures the uncertainty (or randomness) in  $p$ ; a fair coin has high entropy, while a coin that always lands heads has zero entropy. It is also the smallest average number of bits needed to encode outcomes drawn from  $p$ .

*In machine learning:* the entropy of a model's predicted distribution measures its confidence (low entropy means confident, high entropy means unsure). Decision trees also use entropy to choose splits. For a one-hot label, the entropy of the true distribution is 0, since there is no uncertainty about the correct answer.

**(c) Cross-entropy  $H(p, q)$ .** *In general:* cross-entropy is the average surprise when events actually follow the true distribution  $p$ , but you measure surprise using a *different* distribution  $q$ :  $H(p, q) = -\sum_k p_k \log q_k$ . It is the average number of bits you spend if you use a code designed for  $q$  while the data really comes from  $p$ . It is always at least  $H(p)$ , with equality only when  $q = p$ .

*In machine learning:*  $p$  is the true label distribution and  $q = \hat{y}$  is the model's softmax output. Cross-entropy then *is* the classification loss, and the better the model, the smaller the cross-entropy.

**(d) KL divergence  $KL(p \parallel q)$ .** *In general:* KL divergence is the *extra* cost caused by using  $q$  instead of  $p$ :  $KL(p \parallel q) = H(p, q) - H(p)$ . It is never negative, it is zero only when  $p = q$ , and it is not symmetric, so it is a “mismatch score” rather than a true distance.

*In machine learning:* since the data fixes  $H(p)$ , minimizing cross-entropy and minimizing KL divergence are the *same* optimization. KL divergence also appears in other places, such as knowledge distillation and variational autoencoders.

**Connection to regression.** The same idea applies to regression. If the truth is a Gaussian centered at  $\mu_1$  and the model is a Gaussian centered at  $\mu_2$  (both with variance  $\sigma^2$ ), then the KL divergence equals  $\frac{(\mu_1 - \mu_2)^2}{2\sigma^2}$ , which is a squared error. So *MSE is also a cross-entropy or KL divergence*, just between Gaussians; cross-entropy for classification and MSE for regression are the same principle applied to different distributions.

**6.1 Surprise.** The surprise of an event with probability  $p$  is  $-\log_2 p$ .

- Compute the surprise of: a fair coin landing heads ( $p = 0.5$ ); a rare event with  $p = 0.01$ ; a certain event with  $p = 1$ .
- Explain in words why rarer events carry more surprise.

**6.2 Entropy** is the *average* surprise:

$$H(p) = -\sum_k p_k \log_2 p_k.$$

Compute  $H$  for:

- a fair coin (0.5, 0.5)
- a biased coin (0.9, 0.1)
- a three-outcome distribution (0.5, 0.25, 0.25)
- a certain outcome (1, 0) (take  $0 \log 0 = 0$ )

Rank them by entropy. What does high entropy tell you about a distribution?

**6.3 Cross-entropy.** If reality follows  $p$  but you use  $q$  to describe it,

$$H(p, q) = -\sum_k p_k \log_2 q_k.$$

Let the truth be  $p = (0.5, 0.5)$ .

- Compute  $H(p, q)$  with  $q = (0.5, 0.5)$ . How does this compare with  $H(p)$ ?
- Compute  $H(p, q)$  with  $q = (0.9, 0.1)$ .
- Which  $q$  is a better model of  $p$ ? How do the numbers show this?

**6.4 KL divergence.**

$$\text{KL}(p \parallel q) = \sum_k p_k \log_2 \frac{p_k}{q_k}.$$

- (a) Show algebraically that  $\text{KL}(p \parallel q) = H(p, q) - H(p)$ .
- (b) Compute  $\text{KL}(p \parallel q)$  for  $p = (0.5, 0.5)$ ,  $q = (0.9, 0.1)$ .
- (c) Compute  $\text{KL}(q \parallel p)$ . Is KL symmetric?
- (d) What is  $\text{KL}(p \parallel p)$ ? What does this tell you about KL as a “mismatch score”?

*Hint for (a):* Split  $\log(p/q)$  into  $\log p - \log q$  and separate the sums.

**6.5 The key insight.** In training, the true distribution  $p$  comes from the data, so  $H(p)$  is a fixed number that does not depend on the model. Explain why *minimizing cross-entropy  $H(p, q)$  is equivalent to minimizing  $\text{KL}(p \parallel q)$* .

**Part 7: Classification Leads to Cross-Entropy**

**Goal:** Connect Parts 3, 4, and 6.

**Problem statement.**

In classification, the label of each example is a category, so the Gaussian-noise assumption of Part 5 no longer makes sense (a label “cat” cannot be “cat plus 0.3”). Instead, we assume the label is drawn from a Bernoulli distribution (two classes) or a Categorical distribution ( $K$  classes), whose probabilities are produced by the model: a sigmoid output for binary problems, and a softmax output for multi-class problems.

**Connection to machine learning.**

Writing the likelihood of the labels under this model and taking the negative logarithm gives the **binary cross-entropy** or **categorical cross-entropy** loss. So two apparently different explanations agree: the loss is (i) the negative log-likelihood of the labels and (ii) the cross-entropy between the true label distribution and the predicted distribution from Part 6. This part asks you to verify that these two views give the same formula, and to fill in a table that places regression and classification side by side.

**7.1 Binary case.** For a single example with label  $y \in \{0, 1\}$  and predicted probability  $\hat{y}$ :

- (a) Using the Bernoulli formula from 3.1 (with  $p = \hat{y}$ ), write the likelihood.
- (b) Take  $-\log$  to obtain the **binary cross-entropy** loss:

$$-\left[ y \log \hat{y} + (1 - y) \log(1 - \hat{y}) \right].$$

**7.2 Multi-class case.** With one-hot  $\mathbf{y}$  and softmax output  $\hat{\mathbf{y}}$ :

- (a) Using the Categorical formula from 3.2, write the likelihood of one example.
- (b) Take  $-\log$  and show that you obtain  $-\sum_k y_k \log \hat{y}_k$ .
- (c) For  $n$  independent examples, write the total negative log-likelihood.

**7.3 One-hot simplification.** If the true class is  $c$ , show that cross-entropy reduces to  $-\log \hat{y}_c$ . Compute this loss (natural log) for  $\hat{y}_c = 0.9, 0.5, 0.1, 0.001$ . What happens as  $\hat{y}_c \rightarrow 0$ ?

**7.4 Connect the dots.** Fill in the blanks and be ready to explain each row:

|                                        | Regression | Classification |
|----------------------------------------|------------|----------------|
| Assumed distribution of $y$            | ?          | ?              |
| Likelihood of one example              | ?          | ?              |
| $-\log$ likelihood (constants dropped) | ?          | ?              |
| Loss name                              | ?          | ?              |

## Part 8: Why Not MSE for Classification?

**Goal:** Understand the practical failure of MSE with softmax or sigmoid.

**Problem statement.** A natural question from a beginner is: “The model outputs probabilities and the labels are 0 or 1, so why not just compute  $(y - \hat{y})^2$  as in regression?” It is not *forbidden* to do so, and it can sometimes work, but it is a poor choice for two reasons.

**Reason 1 (modelling).** MSE assumes Gaussian noise around a real-valued prediction. Class labels are categories, so this assumption does not match the data. Cross-entropy, in contrast, is derived from the correct (Bernoulli or Categorical) assumption.

**Reason 2 (optimization).** A sigmoid or softmax output *saturates*: when the model is very confident, the output is nearly 0 or 1 and the derivative of the sigmoid is nearly zero. With squared error, this small derivative multiplies the gradient, so a model that is *confidently wrong* receives an almost-zero gradient and cannot correct itself. With cross-entropy, the logarithm exactly cancels this small derivative, and the gradient with respect to the logit becomes simply  $\hat{y} - y$ , which is large precisely when the error is large.

The questions below let you verify both reasons with numbers.

**8.1 Penalty comparison.** The true class is 1 (one-hot  $\mathbf{y} = (1, 0, 0)$ ). Consider the confident wrong prediction  $\hat{\mathbf{y}} = (0.001, 0.499, 0.5)$ . Compute:

- (a) the cross-entropy (natural log)
- (b) the MSE-style loss  $\sum_k (y_k - \hat{y}_k)^2$

Which loss punishes the confident mistake more? By roughly how much?

*Hint:* The squared error for a one-hot target can never exceed 2, while  $-\log \hat{y}$  can grow without limit.

**8.2 The vanishing gradient (binary case).** Take a single logit  $z$ , with  $\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$  and true label  $y = 1$ . The sigmoid satisfies  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ .

- (a) For cross-entropy  $L = -\log \hat{y}$ , show that  $\frac{dL}{dz} = \hat{y} - 1$ .
- (b) For squared error  $L = (1 - \hat{y})^2$ , show that  $\frac{dL}{dz} = 2(\hat{y} - 1)\hat{y}(1 - \hat{y})$ .
- (c) Suppose the model is **confidently wrong**:  $z = -6$ , so  $\hat{y} \approx 0.0025$ . Compute both gradients. Which is larger, and by roughly what factor?
- (d) In 3 to 4 sentences, explain what this means for a network trying to learn from its worst mistakes.

*Hint for (a):* Use the chain rule  $\frac{dL}{dz} = \frac{dL}{d\hat{y}} \frac{d\hat{y}}{dz}$ . For  $L = -\log \hat{y}$ ,  $\frac{dL}{d\hat{y}} = -\frac{1}{\hat{y}}$ .

**8.3 Summary.** Write a short paragraph (5 to 6 sentences) answering the original question: “*Why can we not just use MSE for classification?*” Your answer must mention (i) the probabilistic assumption and (ii) the gradient behavior.

## Part 9 (Optional, Coding): Check It Numerically

Use Python with NumPy.

Deriving a formula on paper and seeing it work on data are different experiences. In this part, you will check every major claim of the assignment numerically: that the peak of the likelihood is at the sample mean, that the Gaussian NLL and MSE share the same minimizer, that  $KL = CE - H$ , and that MSE gradients vanish when a sigmoid saturates while cross-entropy gradients do not.

- 9.1** Simulate 1000 coin flips with true  $p = 0.3$ . Plot the log-likelihood over  $p \in (0, 1)$ . Confirm that its peak is near the sample mean.
- 9.2** Generate data  $y = 2x + 1 + \text{noise}$  (Gaussian,  $\sigma = 0.5$ ). Compute the Gaussian negative log-likelihood and the MSE over a grid of slopes  $w$ . Show that both are minimized at the same  $w$ .
- 9.3** Implement entropy, cross-entropy, and KL divergence for discrete distributions. Verify numerically that  $KL = CE - H$ .
- 9.4** Plot the gradient magnitudes from 8.2 as a function of  $z$  for both losses.

## Submission Checklist

- Every derivation shows intermediate steps.
- Tables in 4.1, 6.2, 7.3, and 7.4 are filled in.
- The three short written explanations (5.3, 6.5, 8.3) are in your own words.
- Any hints you used are marked with “(H)” next to the answer.

## Suggested Grading (100 points)

| Part                                   | Points |
|----------------------------------------|--------|
| 1 to 2: Probability foundations        | 10     |
| 3: Distributions                       | 10     |
| 4: Likelihood and MLE                  | 20     |
| 5: Gaussian to MSE                     | 20     |
| 6: Entropy, CE, KL                     | 20     |
| 7 to 8: Classification and MSE failure | 20     |
| Part 9 (bonus)                         | +10    |